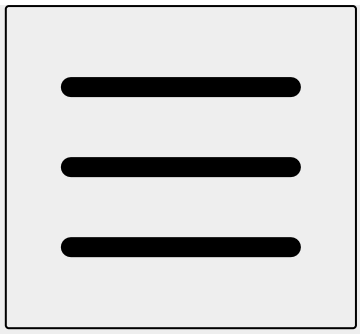


Submodules

config

evaluation

feature_extraction

io

model_training

pipeline

visualization

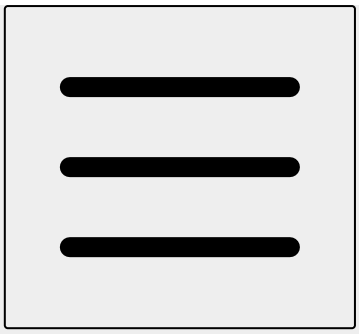
API Documentation

built with pdoc

src

Source package for Conformance Predictions.

- View Source



config.py

src

API Documentation

class Discovery_alg

- ALPHA
- INDUCTIVE
- GENETIC

class ModelRepresentation

- PROCESS_TREE
- BPMN
- PETRI_NET

class PredictionTarget

- PROCESS_OUTCOME
- REMAINING_TIME
- FUTURE_DEVIATIONS

class PredictionFeature

- CONTROL_FLOW
- DIAGNOSTICS
- CASE_ATTRIBUTES
- HYBRID

class RegressionAlgorithm

- DECISION_TREE
- RANDOM_FOREST
- GRADIENT_BOOSTING

class ClassificationAlgorithm

DECISION_TREE

LOGISTIC_REGRESSION

GRADIENT_BOOSTING

built with pdoc

src.config

Configuration module that holds static enum classes for better code interoperability between modules.

•View Source

class Discovery_alg(enum.Enum): •View Source

ALPHA = <Discovery_alg.ALPHA: 'ALPHA'>

INDUCTIVE = <Discovery_alg.INDUCTIVE: 'INDUCTIVE'>

GENETIC = <Discovery_alg.GENETIC: 'GENETIC'>

class ModelRepresentation(enum.Enum): •View Source

PROCESS_TREE = <ModelRepresentation.PROCESS_TREE: 'PROCESS_TREE'>

BPMN = <ModelRepresentation.BPMN: 'BPMN'>

PETRI_NET = <ModelRepresentation.PETRI_NET: 'PETRI_NET'>

class PredictionTarget(enum.Enum): •View Source

PROCESS_OUTCOME = <PredictionTarget.PROCESS_OUTCOME: 'PROCESS_OUTCOME'>

REMAINING_TIME = <PredictionTarget.REMAINING_TIME: 'REMAINING_TIME'>

FUTURE_DEVIATIONS = <PredictionTarget.FUTURE_DEVIATIONS: 'FUTURE_DEVIATIONS'>

class PredictionFeature(enum.Enum): •View Source

CONTROL_FLOW = <PredictionFeature.CONTROL_FLOW: 'CONTROL_FLOW'>

DIAGNOSTICS = <PredictionFeature.DIAGNOSTICS: 'DIAGNOSTICS'>

CASE_ATTRIBUTES = <PredictionFeature.CASE_ATTRIBUTES: 'CASE_ATTRIBUTES'>

HYBRID = <PredictionFeature.HYBRID: 'HYBRID'>

class RegressionAlgorithm(enum.Enum): •View Source

DECISION_TREE = <RegressionAlgorithm.DECISION_TREE: 'DECISION_TREE'>

RANDOM_FOREST = <RegressionAlgorithm.RANDOM_FOREST: 'RANDOM_FOREST'>

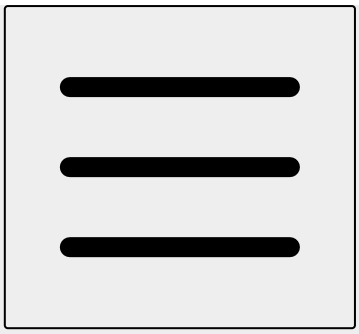
GRADIENT_BOOSTING = <RegressionAlgorithm.GRADIENT_BOOSTING: 'GRADIENT_BOOSTING'>

class ClassificationAlgorithm(enum.Enum): •View Source

DECISION_TREE = <ClassificationAlgorithm.DECISION_TREE: 'DECISION_TREE'>

LOGISTIC_REGRESSION = <ClassificationAlgorithm.LOGISTIC_REGRESSION: 'LOGISTIC_REGRESSION'>

GRADIENT_BOOSTING = <ClassificationAlgorithm.GRADIENT_BOOSTING: 'GRADIENT_BOOSTING'>



IO.py

src

API Documentation

load_event_log()

load_process_model()

discover_process_model()

built with pdoc

src.io

The input module handles the loading of event logs and process models, as well as model discovery from event logs.

•View Source

```
def load_event_log(file_path: str) -> pandas.DataFrame:
```

•View Source

Parses event log data from XES or CSV files into a Dataframe.

Arguments:

- **file_path (str):** Path to file.

Raises:

- **ValueError:** Unsupported log format.

Returns:

pd.DataFrame: Internal event log representation.

```
def load_process_model(
    file_path: str
) -> Tuple[pm4py.objects.petri_net.obj.PetriNet, pm4py.objects.petri_net.obj.Marking, pm4py.objec
```

•View Source

Loads a Petri net or BPMN representation from a file. If BPMN, it is transformed to a Petri Net.

Arguments:

- **file_path (str):** Path to model.

Raises:

- **ValueError:** Unsupported model format.

Returns:

Tuple[PetriNet, Marking, Marking]: Internal petri net description.

```
def discover_process_model(
    log: pandas.DataFrame,
    discovery_alg: src.config.Discovery_alg,
    **kwargs
) -> Tuple[pm4py.objects.petri_net.obj.PetriNet, pm4py.objects.petri_net.obj.Marking, pm4py.objec
```

•View Source

Discovers a Petri net from event log data using Inductive / Alpha / Genetic Miner.

Arguments:

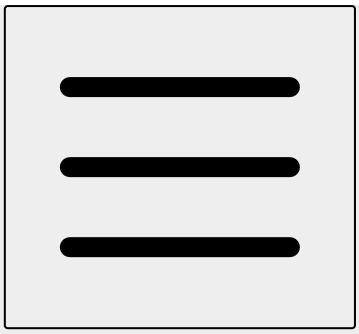
- **log (pd.DataFrame):** The event log Dataframe.
- **discovery_alg (Discovery_alg):** The user selected process discovery algorithm.

Raises:

- **TypeError:** Unsupported discovery algorithm parameters or discovery algorithm.

Returns:

Tuple[PetriNet, Marking, Marking]: Discovered model as internal Petri Net representation.



feature_extraction.py

src

API Documentation

compute_alignments()

generate_prefixes()

extract_remaining_time()

extract_temporal_features()

real_deviation_cost()

extract_label()

list_case_attributes()

summarize_case_attributes()

extract_features()

preprocess_for_ml()

built with pdoc

src.feature_extraction

This module is responsible for extracting features from input data such as

1. Log (pandas dataframe): case attributes, control flow, conformance insights i.e. alignments
2. Model (petri net): conformance insights i.e. alignments

This data is also preprocesed for ML by applying scaling and ensuring that the extracted label- and feature-vectors for the train and test sets are not missing any cells.

View Source

```
def compute_alignments(
    log: pandas.DataFrame,
```

```
    model: Tuple[pm4py.objects.petri_net.obj.PetriNet, pm4py.objects.petri_net.obj.Marking, pm4
) -> List[Dict[str, Any]]:                                                    •View Source
```

Computes optimal alignments between the log and the model (Petri Net).

Ordering matters: pm4py does NOT guarantee that the returned alignment list follows the case order of `log[...].unique()` (passing a DataFrame sorts cases by id internally). Every consumer here pairs alignments to cases positionally, so we convert to an EventLog once, key each alignment by its case id, and re-emit in appearance order — making the positional pairing correct regardless of pm4py's internal ordering.

Arguments:

- **log (pd.DataFrame):** Event log.
- **model (Tuple[PetriNet, Marking, Marking]):** Petri net model (net, im, fm).

Returns:

List[Dict[str, Any]]: The standard PM4Py alignment output (list of dictionaries), ordered to match `log[case_id].unique()` (first-appearance order).

```
def generate_prefixes(
    log: pandas.DataFrame,
    min_length: int = 1,
    max_length: int | None = None,
    step: int = 1,
    gate_activities: List[str] | None = None,
    case_id_key: str = 'case:concept:name',
    timestamp_key: str = 'time:timestamp',
    activity_key: str = 'concept:name'
) -> Tuple[pandas.DataFrame, pandas.DataFrame]:                               •View Source
```

Expand each case into varying-length prefix pseudo-cases (LL-07.1).

For each case and each k in range(min_length, max_length+1, step) with k <= case length, the first k events (time-ordered) become a pseudo-case whose id is f"{orig}@@k{k}". This lets all existing feature/alignment/label code (which keys off case_id_key) run unchanged — each prefix simply looks like another case.

Leakage gating: if gate_activities is given (PROCESS_OUTCOME), only prefixes that end strictly before the first occurrence of any gate activity are emitted, so the outcome is never already inside the prefix.

Arguments:

- **log (pd.DataFrame):** The original event log.
- **min_length (int, optional):** Minimum prefix length. Defaults to 1.
- **max_length (Optional[int], optional):** Maximum prefix length. Defaults to None.
- **step (int, optional):** Step size between prefix lengths. Defaults to 1.
- **gate_activities (Optional[List[str]], optional):** Activities that gate prefix generation. Defaults to None.
- **case_id_key (str, optional):** Key for case ID. Defaults to "case:concept:name".
- **timestamp_key (str, optional):** Key for timestamp. Defaults to "time:timestamp".
- **activity_key (str, optional):** Key for activity name. Defaults to "concept:name".

Returns:

Tuple[pd.DataFrame, pd.DataFrame]: A tuple containing: prefix_log: event-level DataFrame, case_id_key replaced by the unique prefix id, every other column (events, case attributes, timestamp) preserved. prefix_index: DataFrame [prefix_id, orig_case_id, prefix_length].

```
def extract_remaining_time(
        full_log: pandas.DataFrame,
        prefix_log: pandas.DataFrame,
        prefix_index: pandas.DataFrame,
        case_id_key: str = 'case:concept:name',
        timestamp_key: str = 'time:timestamp'
) -> pandas.Series:                                    •View Source
```

Per-prefix remaining time in seconds — the regression label for REMAINING_TIME.

Unlike the classification targets (one label per case, broadcast to all prefixes), remaining time differs for every prefix: it is the time from the END of the observed prefix to the completion of the original case.

```
remaining = (last timestamp of the FULL original case)
          − (last timestamp inside the prefix)
```

The case-end is taken from full_log so the future the model must predict is never inside the features; "now" is the prefix's last event.

Arguments:

- **full_log (pd.DataFrame):** Complete event log (original case ids).
- **prefix_log (pd.DataFrame):** Prefix event log from generate_prefixes; its case_id_key column already holds the unique prefix id.

- **prefix_index (pd.DataFrame):** [prefix_id, orig_case_id, prefix_length] from generate_prefixes.
- **case_id_key (str, optional):** Key for case ID. Defaults to "case:concept:name".
- **timestamp_key (str, optional):** Key for timestamp. Defaults to "time:timestamp".

Returns:

pd.Series: Series indexed by prefix_id, name "remaining_time_seconds" (float).

```
def extract_temporal_features(
        prefix_log: pandas.DataFrame,
        case_id_key: str = 'case:concept:name',
        timestamp_key: str = 'time:timestamp'
) -> pandas.DataFrame:                                    •View Source
```

Per-prefix temporal predictors for remaining-time prediction.

These are the standard, high-impact time signals for remaining-time models.

Arguments:

- **prefix_log (pd.DataFrame):** Prefix event log.
- **case_id_key (str, optional):** Key for case ID. Defaults to "case:concept:name".
- **timestamp_key (str, optional):** Key for timestamp. Defaults to "time:timestamp".

Returns:

pd.DataFrame: DataFrame indexed by prefix id with columns: tf_elapsed_s: time from the prefix's first to last event (how long it has run) tf_since_prev_s: gap between the last two events of the prefix (0 if one event) tf_n_events: number of events observed so far

```
def real_deviation_cost(alignment_moves: List[Tuple[Any, Any]]) -> int:    •View Source
```

Count the REAL conformance deviations in one alignment.

A pm4py alignment move is a (log_move, model_move) tuple: (a, a) synchronous move -> 0 (conforming) (a, '>>') log move: event a not allowed by the model here -> +1 ('>>', a) model move on a VISIBLE transition a (expected, missing) -> +1 ('>>', None) model move on an invisible/silent (tau) transition -> 0 (routing)

Silent/tau moves are normal routing in (e.g.) Inductive-Miner models and carry zero cost; counting them as deviations is exactly the bug this helper avoids. Mirrors the move classification used for DIAGNOSTICS features in extract_features.

Arguments:

- **alignment_moves (List[Tuple[Any, Any]]):** List of alignment moves.

Returns:

int: Total cost of real deviations.

```
def extract_label(
    log: pandas.DataFrame,
    target_type: src.config.PredictionTarget,
    outcome_activities: List[str] | None = None,
    alignments: List[Dict[str, Any]] | None = None,
    deviation_cost_threshold: int = 0
) -> pandas.Series:
```

•View Source

Derives labels for each case.

- PROCESS_OUTCOME: Checks if any activity in 'outcome_activities' exists in the trace.
- FUTURE_DEVIATIONS: label = 1 if the case's real deviation cost exceeds deviation_cost_threshold (HL-08). Real cost ignores silent/tau routing moves.
- REMAINING_TIME: per-prefix regression label (see extract_remaining_time).

Arguments:

- **log (pd.DataFrame):** Event log.
- **target_type (PredictionTarget):** Type of prediction target.
- **outcome_activities (Optional[List[str]], optional):** Activities defining outcomes. Defaults to None.
- **alignments (Optional[List[Dict[str, Any]]], optional):** Alignment results. Defaults to None.
- **deviation_cost_threshold (int, optional):** Threshold for future deviations. Defaults to 0.

Raises:

- **ValueError:** If required arguments for target types are missing.
- **NotImplementedError:** If target type is not supported.

Returns:

pd.Series: Series of labels indexed by case ID.

```
def list_case_attributes(log: pandas.DataFrame) -> List[str]:
```

•View Source

Return the case-level (trace) attribute column names present in the log.

A column is treated as case-level if it is either explicitly prefixed 'case:' (as PM4Py does for CSV logs) or — for XES logs, where PM4Py flattens trace attributes into unprefixed columns mixed with event attributes — its value is constant within every case. Standard event/ system columns (concept:name, time:timestamp, lifecycle:transition, ...) are always excluded.

Arguments:

- **log (pd.DataFrame):** Event log.

Returns:

List[str]: List of case-level attribute column names.

```
def summarize_case_attributes(log: pandas.DataFrame) -> pandas.DataFrame:
```

•View Source

Small summary table for CLI display of case-level attributes.

Helps the analyst choose meaningful attributes (HL-03/HL-04).

Arguments:

- **log (pd.DataFrame):** Event log.

Returns:

pd.DataFrame: Summary table with columns [attribute, dtype, n_unique, example].

```
def extract_features(
    log: pandas.DataFrame,
    feature_type: src.config.PredictionFeature,
    model: Tuple | None = None,
    alignments: List[Dict[str, Any]] | None = None,
    selected_case_attributes: List[str] | None = None
) -> pandas.DataFrame:
```

•View Source

Extracts features based on the specified feature type.

- CONTROL_FLOW: Extracts the trace prefix (Bag-of-activities).
- DIAGNOSTICS: Extracts the alignment-based trace prefix diagnostics (Bag-of-move-types).
- CASE_ATTRIBUTES: Extracts static attributes about the case.
- HYBRID: Combines the active feature sets.

Arguments:

- **log (pd.DataFrame):** Event log.
- **feature_type (PredictionFeature):** Type of features to extract.
- **model (Optional[Tuple], optional):** Process model. Defaults to None.
- **alignments (Optional[List[Dict[str, Any]]], optional):** Alignment results. Defaults to None.
- **selected_case_attributes (Optional[List[str]], optional):** Subset of case attributes. Defaults to None.

Raises:

- **ValueError:** If required data for feature type is missing or if feature type is unsupported.

Returns:

pd.DataFrame: Feature matrix indexed by case ID.

```
def preprocess_for_ml(
    X_raw: pandas.DataFrame,
    y_raw: pandas.Series,
    is_regression: bool = False
) -> Tuple[pandas.DataFrame, numpy.ndarray, List[str]]:
```
•View Source

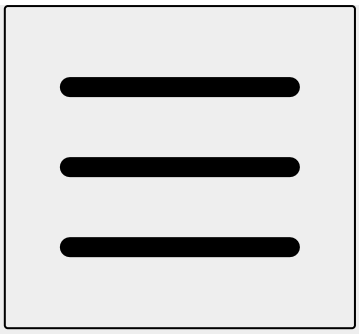
Encodes and scales the raw data for ML training.

Arguments:

- **X_raw (pd.DataFrame):** Raw feature matrix.
- **y_raw (pd.Series):** Raw label vector.
- **is_regression (bool, optional):** When True the target is continuous (e.g. remaining time in seconds) — it is kept numeric and NOT label-encoded. Defaults to False.

Returns:

Tuple[pd.DataFrame, np.ndarray, List[str]]: A tuple containing: - X: feature DataFrame (columns preserved for downstream alignment). - y: encoded label vector (classification) or float vector (regression). - class_names: ordered list of original class labels for classification, [] for regression.



model_training.py

src

API Documentation

predict()

create_classification_pipeline()

train_classification_model()

create_regression_pipeline()

train_regression_model()

predict_regression()

built with pdoc

src.model_training

Module for training and applying predictive models using scikit-learn. Supported are

1. Regression:
    - Random Forest
    - Decision Tree
    - Gradient Boosting
2. Regression:
    - Logistic regression
    - Decision Tree
    - Gradient Boosting predictors. Preprocessed training data is used to train these predictors and targets can be predicted using feature data from the test split.

• View Source

```
def predict(
    model: sklearn.pipeline.Pipeline,
    X: pandas.DataFrame
```

```
) -> Tuple[numpy.ndarray, numpy.ndarray]:                    •View Source
```

Applies the fitted model to generate predictions.

Arguments:

- **model (Pipeline):** A fitted scikit-learn Pipeline.
- **X (pd.DataFrame):** Feature matrix for which to generate predictions.

Returns:

Tuple[np.ndarray, np.ndarray]: Tuple containing hard predictions (0/1), probabilities for the positive class.

```
def create_classification_pipeline(
    algorithm: src.config.ClassificationAlgorithm = <ClassificationAlgorithm.DECISION_TREE: 'DE
    ml_params: Dict[str, Any] | None = None
) -> sklearn.pipeline.Pipeline:                              •View Source
```

Build a StandardScaler + classifier pipeline for deviation/outcome risk prediction.

Arguments:

- **algorithm (ClassificationAlgorithm, optional):** User specified classification algorithm (Logistic Regression / Gradient Boosting / Decision Tree). Defaults to ClassificationAlgorithm.DECISION_TREE.
- **ml_params (Optional[Dict[str, Any]], optional):** Parameters that are to be passed through to the specified classification algorithm. Defaults to None.

Returns:

Pipeline: A Pipeline object modified with the given parameters.

```
def train_classification_model(
    X_train: pandas.DataFrame,
    y_train: numpy.ndarray,
    algorithm: src.config.ClassificationAlgorithm = <ClassificationAlgorithm.DECISION_TREE: 'DE
    ml_params: Dict[str, Any] | None = None
) -> sklearn.pipeline.Pipeline:                              •View Source
```

Creates a pipeline by calling @create_classification_pipeline and then fits the classification pipeline using the provided data.

Arguments:

- **X_train (pd.DataFrame):** Training data: feature dataframe.
- **y_train (np.ndarray):** Training data: target vector.

- **algorithm (ClassificationAlgorithm, optional):** User specified classification algorithm. Defaults to ClassificationAlgorithm.DECISION_TREE.
- **ml_params (Optional[Dict[str, Any]], optional):** User specified parameters for the provided classification algorithm. Defaults to None.

Raises:

- **ValueError:** Raises a ValueError if parameters or algorithm does not fit.

Returns:

Pipeline: A fitted sklearn pipeline object.

```
def create_regression_pipeline(
        algorithm: src.config.RegressionAlgorithm = <RegressionAlgorithm.DECISION_TREE: 'DECISI
        ml_params: Dict[str, Any] | None = None
) -> sklearn.pipeline.Pipeline:                                          •View Source
```

Build a StandardScaler + regressor pipeline for remaining-time prediction.

Arguments:

- **algorithm (RegressionAlgorithm, optional):** User specified regression algorithm. Defaults to RegressionAlgorithm.DECISION_TREE.
- **ml_params (Optional[Dict[str, Any]], optional):** Corresponding regression algorithm parameters. Defaults to None.

Returns:

Pipeline: Sklearn pipeline object modified given the provided parameters.

```
def train_regression_model(
        X_train: pandas.DataFrame,
        y_train: numpy.ndarray,
        algorithm: src.config.RegressionAlgorithm = <RegressionAlgorithm.DECISION_TREE: 'DECISI
        ml_params: Dict[str, Any] | None = None
) -> sklearn.pipeline.Pipeline:                                          •View Source
```

Fit a regression pipeline to the training data (remaining time in seconds).

Arguments:

- **X_train (pd.DataFrame):** Training data: feature dataframe.
- **y_train (np.ndarray):** Training data: target vector.
- **algorithm (RegressionAlgorithm, optional):** User specified regression algorithm. Defaults to RegressionAlgorithm.DECISION_TREE.

- **ml_params (Optional[Dict[str, Any]], optional):** Corresponding regression algorithm parameters. Defaults to None.

Returns:

Pipeline: Sklearn pipeline object fitted to provided training data.

```
def predict_regression(model: sklearn.pipeline.Pipeline,
X: pandas.DataFrame) -> numpy.ndarray:                    •View Source
```

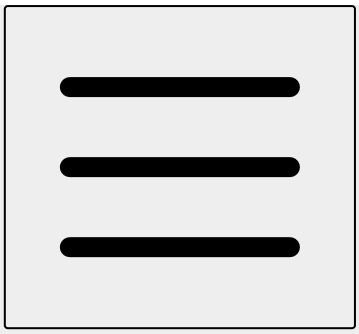
Apply a fitted regression pipeline; returns continuous predictions (no proba).

Arguments:

- **model (Pipeline):** Sklearn pipeline object (specified and fitted).
- **X (pd.DataFrame):** Input data: feature dataframe.

Returns:

np.ndarray: Predicted values vector.



pipeline.py

src

API Documentation

run_conformance_prediction_pipeline()

prepare_log_and_model()

extract_target_and_feature()

extract_target_and_feature_over_prefixes()

built with pdoc

src.pipeline

This module bundles pipelines that perform the relevant steps from input to results. These pipeline fragments are the following:

1. Prepare Log and Model
2. Extract Target and Feature (optionally: for given prefix lenght(s))
3. Run Conformance Prediction (+ compute Evaluation scores)

•View Source

```
def run_conformance_prediction_pipeline(
        X_test: pandas.DataFrame,
        y_test: numpy.ndarray,
        X_train: pandas.DataFrame,
        y_train: numpy.ndarray,
        ml_params: Dict[str, Any] | None = None,
        preloaded_model=None,
        test_prefix_lengths: numpy.ndarray | None = None,
        is_regression: bool = False
) -> Dict[str, Any]:                                    •View Source
```

Main orchestration function for the entire pipeline.

Arguments:

- **X_test (pd.DataFrame):** Feature test dataframe.
- **y_test (np.ndarray):** Target test vector.
- **X_train (pd.DataFrame):** Feature train dataframe.
- **y_train (np.ndarray):** Target train vector.
- **ml_params (Optional[Dict[str, Any]], optional):** User specified machine learning (ML) parameters. Defaults to None.
- **preloaded_model (_type_, optional):** Already existing model - if provided, the this model is used instead of training a new one. Defaults to None.
- **test_prefix_lengths (Optional[np.ndarray], optional):** Prefix lengths to test. Defaults to None.
- **is_regression (bool, optional):** Wether this pipeline is regression or classification. Defaults to False.

Returns:

Dict[str, Any]: A dictionary with the trained model and other information such as the predictions and target probabilities.

```
def prepare_log_and_model(
        log_path: str,
        model_path: str | None = None,
        discovery_alg: src.config.Discovery_alg | None = None,
        discovery_params: Dict[str, Any] | None = None,
        vis_params: Dict[str, Any] | None = None
) -> Tuple[Tuple[pm4py.objects.petri_net.obj.PetriNet, pm4py.objects.petri_net.obj.Marking, pm4py
```

•View Source

Orchestrates the loading of data, model acquisition (load or discovery), and optional visualization.

Arguments:

- **log_path (str):** Path to the event log.
- **model_path (Optional[str], optional):** Path to the model. Defaults to None.
- **discovery_alg (Optional[config.Discovery_alg], optional):** User specified discovery algorithm. Defaults to None.
- **discovery_params (Optional[Dict[str, Any]], optional):** Corresponding parameters for the discovery algorithm. Defaults to None.
- **vis_params (Optional[Dict[str, Any]], optional):** Parameters for the initial model visualization. Defaults to None.

Raises:

- **ValueError:** _description_

Returns:

Tuple[Tuple[PetriNet, Marking, Marking], pd.DataFrame]: _description_

```
def extract_target_and_feature(
    target: src.config.PredictionTarget,
    feature: src.config.PredictionFeature,
    model: Tuple[Any, Any, Any],
    log: pandas.DataFrame,
    outcome_activities: List[str] | None = None,
    prefix_length: int | None = None,
    selected_case_attributes: List[str] | None = None,
    deviation_cost_threshold: int = 0
) -> Tuple[pandas.DataFrame, numpy.ndarray, List[str]]:
```

•View Source

Orchestrates the extraction of labels (y) and features (X). Ensures that the features and targets are aligned by case/prefix.

Arguments:

- **target (config.PredictionTarget):** User specified target of prediction.
- **feature (config.PredictionFeature):** User specified features basis.
- **model (Tuple[Any, Any, Any]):** Internal (PM4Py) Petri Net representation of the process model.
- **log (pd.DataFrame):** Internal event log representation as dataframe.
- **outcome_activities (Optional[List[str]], optional):** User specified outcome activities (only necessary if 'process outcome' is selected as prediction target). Defaults to None.
- **prefix_length (Optional[int], optional):** User specified length of prefixes that are to be extracted from traces. Defaults to None.
- **selected_case_attributes (Optional[List[str]], optional):** user specified case attributes (only necessary if case attributes are to be part of the feature basis). Defaults to None.
- **deviation_cost_threshold (int, optional):** Deviation cost threshhold (only necessary for prediction type of remaining time). Defaults to 0.

Returns:

Tuple[pd.DataFrame, np.ndarray, List[str]]: Extracted feature dataframe and target vector as well as target class names.

```
def extract_target_and_feature_over_prefixes(
    target: src.config.PredictionTarget,
```

```
        feature: src.config.PredictionFeature,
        model: Tuple[Any, Any, Any],
        log: pandas.DataFrame,
        outcome_activities: List[str] | None = None,
        selected_case_attributes: List[str] | None = None,
        min_length: int = 1,
        max_length: int | None = None,
        step: int = 1,
        gate_outcome: bool = True,
        deviation_cost_threshold: int = 0
) -> Tuple[pandas.DataFrame, numpy.ndarray, List[str], numpy.ndarray]:      •View Source
```

Prefix-level (pooled) variant of extract_target_and_feature.

Expands each case into varying-length prefixes (exactly-k => min==max==k; varying 1..k => min=1, max=k), pools them into one (X, y) dataset, and returns each row's prefix length as metadata for by-length analysis.

Labels are computed once on the FULL log per original case and broadcast to that case's prefixes; for PROCESS_OUTCOME, prefixes are gated to end before the first outcome activity so the label is never inside the features.

REMAINING_TIME is the exception: it is a per-prefix regression label (seconds from each prefix's end to case completion), computed per prefix rather than broadcast, and explicit temporal features (elapsed / since-previous / event-count) are appended.

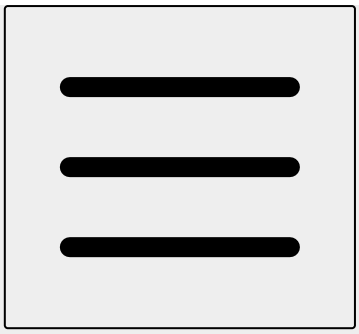
Arguments:

- **target (config.PredictionTarget):** User specified target of prediction.
- **feature (config.PredictionFeature):** User specified features basis.
- **model (Tuple[Any, Any, Any]):** Internal (PM4Py) Petri Net representation of the process model.
- **log (pd.DataFrame):** Internal event log representation as dataframe.
- **outcome_activities (Optional[List[str]], optional):** User specified outcome activities (only necessary if 'process outcome' is selected as prediction target). Defaults to None.
- **prefix_length (Optional[int], optional):** User specified length of prefixes that are to be extracted from traces. Defaults to None.
- **selected_case_attributes (Optional[List[str]], optional):** user specified case attributes (only necessary if case attributes are to be part of the feature basis). Defaults to None.
- **deviation_cost_threshold (int, optional):** Deviation cost threshhold (only necessary for prediction type of remaining time). Defaults to 0.

- **min_length (int, optional):** Lower bound of prefix lengths. Defaults to 1.
- **max_length (Optional[int], optional):** Lower bound of prefix lengths. Defaults to None.
- **step (int, optional):** Step interval between lower and upper bound. Defaults to 1.
- **gate_outcome (bool, optional):** Defaults to True.

Returns:

Tuple[pd.DataFrame, np.ndarray, List[str]]: Extracted feature dataframe and target vector as well as target class names and prefix_lengths (prefix_lengths row-aligned to X).



evaluation.py

src

API Documentation

measure_runtime()

measure_memory()

compute_regression_metrics()

compute_classification_metrics()

compute_roc_curve()

compute_pr_curve()

prepare_curve_data()

metrics_by_prefix_length()

evaluate_by_prefix_length()

compare_models()

compute_mse()

compute_rmse()

compute_mae()

built with pdoc

src.evaluation

Evaluation module for the predictive monitoring pipeline.

This module is responsible for (list not guaranteed to be complete):

1. Infrastructure utilities (called during the execution of our CLI):
    - measure_runtime / measure_memory: wrappers around (relatively big) pipeline fragments

2. Reporting (called by pipeline fragments and CLI after predictions are collected):
    - compute_classification_metrics: standard sklearn metrics for one model run
    - compute_regression_metrics: standard metrics for one model run
    - evaluate_by_prefix_length: metrics of models masked by a prefix of given length

•View Source

```
def measure_runtime(func: Callable, *args: Any, **kwargs: Any) -> Tuple[Any, float]:
```

•View Source

Measure the runtime of an arbitrary function.

Arguments:

- **func (Callable):** The Python function whose runtime is to be measured.

Returns:

Tuple[Any, float]: The return value of the passed function and the elapsed time (in seconds).

```
def measure_memory(func: Callable, *args: Any, **kwargs: Any) -> Tuple[Any, float]:
```

•View Source

Measure the peak memory usage of an arbitrary function.

Arguments:

- **func (Callable):** The Python function whose peak memory usage is to be measured.

Returns:

Tuple[Any, float]: The return value of the passed function and the peak RSS in MB.

```
def compute_regression_metrics(y_true: numpy.ndarray,
y_pred: numpy.ndarray) -> Dict[str, float]:
```

•View Source

Aggregation function for all available regression metrics.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.

Returns:

Dict[str, float]: A dictionary containing regression metrics: Root Mean Squared Error ('RMSE'), Mean Squared Error ('MSE'), Mean Absolute Error ('MAE'),

```
def compute_classification_metrics(
    y_true: numpy.ndarray,
```

```
    y_pred: numpy.ndarray,
    y_pred_proba: numpy.ndarray | None = None
) -> Dict[str, float]:                                    •View Source
```

Aggregation of classification metrics.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.
- **y_pred_proba (Optional[np.ndarray], optional):** Class probability. Defaults to None.

Returns:

Dict[str, float]: Dictionary of classification scores such as: Accuracy, Precision, Recall, F1, ROC/AUC, Brier score.

```
def compute_roc_curve(
    y_true: numpy.ndarray,
    y_score: numpy.ndarray,
    pos_label=None
) -> Tuple[numpy.ndarray, numpy.ndarray]:                 •View Source
```

Invokes the sklearn.metrics function that calculates the ROC curve.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_score (np.ndarray):** Predicted vector.
- **pos_label (_type_, optional):** Class probability. Defaults to None.

Returns:

Tuple[np.ndarray, np.ndarray]: Lists of false positive and true positive rates (FPR, TPR).

```
def compute_pr_curve(
    y_true: numpy.ndarray,
    y_score: numpy.ndarray,
    pos_label=None
) -> Tuple[numpy.ndarray, numpy.ndarray, float]:          •View Source
```

```
def prepare_curve_data(
    y_true: numpy.ndarray,
```

```
        y_prob: numpy.ndarray,
        class_names: List[str]
) -> Tuple[Dict[str, Tuple[numpy.ndarray, numpy.ndarray]], Dict[str, Tuple[numpy.ndarray, numpy.nd
```

•View Source

Preparation of data for and aggregation of curves (ROC / PR) per class.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_prob (np.ndarray):** Class probability.
- **class_names (List[str]):** Class names.

Returns:

Tuple[Dict[str, Tuple[np.ndarray, np.ndarray]], Dict[str, Tuple[np.ndarray, np.ndarray, float]]]: Tuple of prepared data for ROC and PR.

```
def metrics_by_prefix_length(
        y_true: numpy.ndarray,
        y_pred: numpy.ndarray,
        y_prob: numpy.ndarray | None,
        prefix_lengths: numpy.ndarray,
        is_regression: bool = False,
        min_group_size: int = 10
) -> pandas.DataFrame:
```

•View Source

Slice one pooled model's predictions by prefix length and compute metrics per length.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Prediction vector.
- **y_prob (Optional[np.ndarray]):** Class probability.
- **prefix_lengths (np.ndarray):** Prefix length of each test instance.
- **is_regression (bool, optional):** Determins whether to compute regression or classification metrics. Defaults to False.
- **min_group_size (int, optional):** Groups with fewer samples than specified are dropped. Defaults to 10.

Returns:

pd.DataFrame: Dataframe of metrics by the prefix length. One row per prefix length - metric combination.

```
def evaluate_by_prefix_length(
```

```
    target: src.config.PredictionTarget,
    feature: src.config.PredictionFeature,
    model: Tuple[Any, Any, Any],
    log: pandas.DataFrame,
    outcome_activities: List[str] | None = None,
    prefix_range_start: int | None = None,
    prefix_range_end: int | None = None
) -> pandas.DataFrame:
```
•View Source

Trains models with trace-prefixes of specified length and aggregates the resulting F1 and Accuracy scores.

Arguments:

- **target (config.PredictionTarget):** User specified target type.
- **feature (config.PredictionFeature):** User specified feature type (should include trace, otherwise pointless).
- **model (Tuple[Any, Any, Any]):** Petri net.
- **log (pd.DataFrame):** Event log.
- **outcome_activities (Optional[List[str]], optional):** Optionally specified outcome activities. Required for prediction type 'process outcome'. Defaults to None.
- **prefix_range_start (Optional[int], optional):** Specified lower bound of prefix length range. Defaults to None. If not specified, the maximum of (1, lower quartile trace length) is used.
- **prefix_range_end (Optional[int], optional):** Specified upper bound of prefix length range. Defaults to None. If not specified, the median trace length is identified and used.

Returns:

pd.DataFrame: A dataframe containing the F1 and accuracy score per prefix length in the provided range.

```
def compare_models(
    baseline_results: Dict[int, Dict[str, float]],
    conformance_results: Dict[int, Dict[str, float]]
) -> pandas.DataFrame:
```
•View Source

```
def compute_mse(y_true: numpy.ndarray, y_pred: numpy.ndarray) -> numpy.float64:
```
•View Source

Computation of Mean Squared Error (MSE).

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.

Returns:

np.float64: MSE.

```
def compute_rmse(y_true: numpy.ndarray, y_pred: numpy.ndarray) -> numpy.float64:
```

•View Source

Computation of Root Mean Squared Error (RMSE).

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.

Returns:

np.float64: RMSE.

```
def compute_mae(y_true: numpy.ndarray, y_pred: numpy.ndarray) -> numpy.float64:
```

•View Source

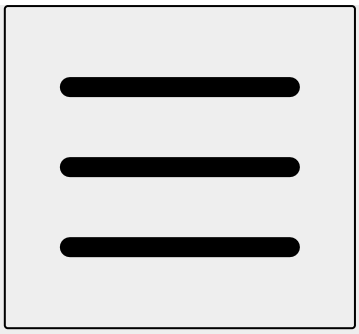
Computation of Mean Absolute Error (MAE).

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.

Returns:

np.float64: MAE.



visualization.py

src

API Documentation

visualize_process_model()

visualize_decision_tree()

plot_confusion_matrix_image()

plot_per_class_bar_chart()

save_evaluation_visuals()

save_curve_visuals()

plot_roc_curves()

plot_pr_curves()

plot_evaluate_by_prefix_length()

plot_processing_times()

plot_runtime_log_size_relation()

plot_feature_importance()

display_confusion_matrix_text()

display_per_class_stats()

built with pdoc

src.visualization

In this module, all visualizations, that can be created for our conformance predictions project, are bundled. This includes a great variety of different visualizations. The are mainly created using matplotlib or built-in visualizations from other libraries that we listed in our requirements engineering document.

• View Source

```
def visualize_process_model(
        model: Tuple[pm4py.objects.petri_net.obj.PetriNet, pm4py.objects.petri_net.obj.Marking, pm4
        output_path: str | None,
        model_representation: src.config.ModelRepresentation
):
```
•View Source

Visualize a process model with PM4Py.

Arguments:

- **model (Tuple[PetriNet, Marking, Marking]):** Petri Net as internal (PM4Py) representation.
- **output_path (Optional[str]):** Path where output of visualizations shall be saved to.
- **model_representation (ModelRepresentation):** Desired model representation (BPMN / Petri Net / Process Tree).

```
def visualize_decision_tree(
        model: sklearn.pipeline.Pipeline,
        feature_names: List[str],
        class_names: List[str],
        output_path: str | None
) -> None:
```
•View Source

Renders the fitted Decision Tree from the pipeline. If output_path is given, saves to that file; otherwise opens an interactive window.

Arguments:

- **model (Pipeline):** Sklearn pipeline object.
- **feature_names (List[str]):** Names of features.
- **class_names (List[str]):** Names of target classes.
- **output_path (Optional[str], optional):** Path where output of visualizations shall be saved to.

```
def plot_confusion_matrix_image(
        y_true: numpy.ndarray,
        y_pred: numpy.ndarray,
        class_names: List[str],
        output_path: str | None
) -> None:
```
•View Source

Plots a atandard confusion matrix and saves it to the specified output path.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.
- **class_names (List[str]):** Class names of targets.
- **output_path (Optional[str]):** Path where output of visualizations shall be saved to.

```
def plot_per_class_bar_chart(
        y_true: numpy.ndarray,
        y_pred: numpy.ndarray,
        class_names: List[str],
        output_path: str | None
) -> None:                                                    •View Source
```

Save a stacked bar chart (green = correct, red = incorrect) per class.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.
- **class_names (List[str]):** Class names of targets.
- **output_path (Optional[str]):** Path where output of visualizations shall be saved to.

```
def save_evaluation_visuals(
        y_true: numpy.ndarray,
        y_pred: numpy.ndarray,
        class_names: List[str],
        base_dir: str | None = None
) -> str:                                                     •View Source
```

Create a timestamped folder and save confusion matrix + bar chart inside it.

Returns the absolute path of the created folder.

```
def save_curve_visuals(
        roc_data: Dict[str, Tuple[numpy.ndarray, numpy.ndarray]],
        pr_data: Dict[str, Tuple[numpy.ndarray, numpy.ndarray, float]],
        base_dir: str | None = None
) -> str:                                                     •View Source
```

Create a timestamped folder and save ROC and PR curves inside it.

Returns the absolute path of the created folder.

```
def plot_roc_curves(
        roc_data: Dict[str, Tuple[numpy.ndarray, numpy.ndarray]],
        output_path: str | None = None
) -> None:
```
•View Source

Plot overlaid ROC curves for multiple models.

Arguments:

- **roc_data (Dict[str, Tuple[np.ndarray, np.ndarray]]):** Mapping of label -> list of (false positive rate, true positive rate) tuples.
- **output_path (Optional[str], optional):** Path where output of visualizations shall be saved to. Defaults to None.

```
def plot_pr_curves(
        pr_data: Dict[str, Tuple[numpy.ndarray, numpy.ndarray, float]],
        output_path: str | None = None
) -> None:
```
•View Source

Plot overlaid Precision-Recall curves for multiple models.

Arguments:

- **pr_data (Dict[str, Tuple[np.ndarray, np.ndarray]]):** Mapping of label -> (precision, recall, average_precision)
- **output_path (Optional[str], optional):** Path where output of visualizations shall be saved to. If None, show interactively.

```
def plot_evaluate_by_prefix_length(results_df: pandas.DataFrame,
output_path: str | None = None) -> None:
```
•View Source

Plots the results of metrics_by_prefix_length. X-axis: prefix_length. Y-axis: Metrics (Accuracy/F1 for clf, MAE for reg).

```
def plot_processing_times(
        time_list: List[Tuple[str, float]],
        output_path: str | None = None
) -> None:
```
•View Source

Creates a pie chart of processing times for different pipeline steps. If output_path is provided, saves to that path; otherwise displays interactively.

Arguments:

- **time_list (List[Tuple[str, float]]):** List of keyed times: (function name, time).
- **output_path (Optional[str], optional):** Path where output of visualizations shall be saved to. Defaults to None.

```
def plot_runtime_log_size_relation(
        log_sizes: List[float],
        runtimes: List[float],
        output_path: str = None
):                                                          •View Source
```

Plots a simple line graph of log size vs runtime.

Arguments:

- **log_sizes (List[float]):** List of log sizes (in MB).
- **runtimes (List[float]):** List of runtimes per log.
- **output_path (str, optional):** Path where output of visualizations shall be saved to. Defaults to None.

```
def plot_feature_importance(
        importances: pandas.Series,
        conformance_feature_names: List[str],
        top_n: int = 20,
        output_path: str | None = None
) -> None:                                                  •View Source
```

```
def display_confusion_matrix_text(
        y_true: numpy.ndarray,
        y_pred: numpy.ndarray,
        class_names: List[str]
) -> None:                                                  •View Source
```

Print a formatted confusion matrix to the terminal.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.
- **class_names (List[str]):** Target class names.

```
def display_per_class_stats(
```

```
        y_true: numpy.ndarray,
        y_pred: numpy.ndarray,
        class_names: List[str]
) -> None:                                                    •View Source
```

Print per-class correct / incorrect percentages to the terminal.

Arguments:

- **y_true (np.ndarray):** Target vector.
- **y_pred (np.ndarray):** Predicted vector.
- **class_names (List[str]):** Target class names.